数据分析中的大模型工具应用[1]

实

践

手

册

[1] 代码仓库：https://github.com/Stepstarr/HFUT-DA-practicum.git

目录

前言 1
API 调用 2
API 调用介绍 2
API 调用示例 3
示例 1:调用基础格式 3
示例 2:正式调用 4
补充：API 调用所需时间和多线程调用 5
实践 6
实践案例：从用户评论中提取关键信息 6
课堂作业：某领域新闻分类 10
大模型微调 12
微调介绍和数据格式 12
实践 13
实践案例：微调一个用户评论信息抽取模型 13
课堂作业：微调模型展示 20
Vibe coding 实践 21
AI IDE 介绍与下载 21
能力初探 21
实践 22
实践案例：简易的用户评论分析器 22
课堂作业：数据分析相关的小工具 24
拓展资料 25
提示词写法 25
提示词基本要素 25
提示词优化 26
模型微调与推理 27
管理学科研领域中的大模型 28
大模型作为工具 28
大模型作为研究对象 31
其他 32
大模型面向特定领域的后训练 32

前言

本实践教程主要面向管理科学领域，旨在提供一套以“大模型”为核心分析工具的实践手册。**针对不同类型的管理问题，学习如何设计相应的分析方案，以有效达成目标。**

为此，该实践手册首先讲解基础知识，涵盖大模型 API 的调用与模型微调，再结合具体案例，展示不同管理场景下的应用方法。通过上述内容，大家将逐步理解大模型的能力边界及其能够完成的任务，**从而在面对实际问题时，能够将大模型作为一个可行的工具选项**。

此外，本课程还增设“Vibe coding”环节，帮助读者借助大模型进行应用开发。希望大家不仅能掌握方法，更能借助大模型的力量，build something great。

在实际上机课程中，建议按照**以下三个步骤**开展实践：Step 1 先完成大模型 API 的基础调用练习，并结合具体案例观察和分析 API 在不同管理场景下的表现与效果；Step 2 尝试对模型进行微调，感受微调前后模型在特定任务上的能力变化与性能提升；Step 3 以“Vibe coding”章节内容为基础，将其作为课程作业，独立开发一套可运行的应用程序或工具。

学有余力的同学可进一步阅读拓展材料。

注：实践教程的代码 90%由大模型生成，在相关代码文件里注释了能生成这些代码的提示词，可供参考。

优先关注任务背景、解决问题的思路，代码实现仅为工具。

API 调用

通过本节，可回答： 什么是大模型 API 调用？ 什么时候用结构化提示词模板输出，为什么要用？ 怎么优化数据分析（基于调用大模型 API 的数据分析）结果。 调用大模型 API 的场景举例。

该章节代码对应位置：[HFUT-DA-practicum /api_examples]

API 调用介绍[2]

大模型 API 介绍

大模型 API（应用程序编程接口）是一种让开发者通过代码向云端大模型发送请求、并获取生成结果的服务。你可以把它理解成“程序之间的对话方式”：你的程序自动构造一条包含指令或问题的消息（例如“请你写一段 python 代码”），通过 HTTP 协议发送给模型服务端，服务端再返回模型生成的内容。API 调用通常需要密钥认证、按请求量或 Token 数计费，并支持批量、自动化、嵌入其他应用（如客服机器人、代码助手、内容审核系统）等场景。

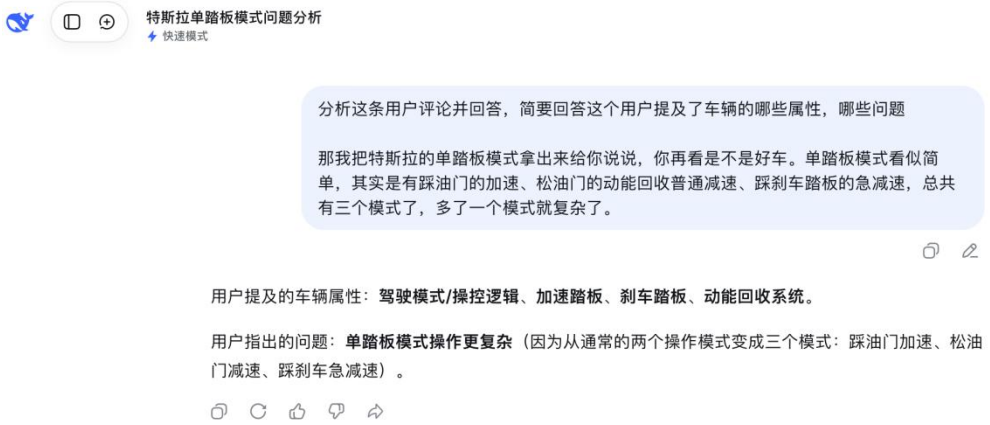


图 1 网页版调用大模型示例

与网络版使用的区别

交互方式：普通用户通过浏览器手动输入文字、点击按钮，看到图形界面返回结果；API 调用则是程序自动发送请求、自动接收解析返回数据，没有界面，适合嵌入系统。

---

[2] *更多信息见：*[deepseek 接口文档]

控制能力：API 允许你精确控制模型参数（如温度、最大长度、停止符等），而网页版通常只暴露少量简单选项。

使用场景：网页适合人工对话、临时查询；**API 适合批量处理、实时集成、自动化流程。**

API 调用示例

示例 1:调用基础格式

代码位置：api_examples/api_example.py

```
import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ.get('DEEPSEEK_API_KEY'),
    base_url="https://api.deepseek.com")

response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "你是一个有用的助手"},
        {"role": "user", "content": "你好，介绍一下你自己"},
    ],
    stream=False
)
print(response.choices[0].message.content)
```

将代码 api_key=os.environ.get('DEEPSEEK_API_KEY')替换为：

api_key=你在平台申请的 key

点击运行，运行结果如下所示：

图 2 调用 API 得到大模型回复结果

示例 2:正式调用

根据示例 1，体验了用代码连接大模型获取回复。在实际应用中，主要对基础格式中以下相应代码片段进行修改：

```
response = client.chat.completions.create(
    model="deepseek-chat",
    messages=[
        {"role": "system", "content": "你是一个有用的助手"},
        {"role": "user", "content": "你好"},
    ],
    stream=False
)
```

1. 设定模型名

例如，deepseek 支持 deepseek-chat、deepseek-reasoner 模型

若要使用 deepseek-reasoner 模型，则设置：

```
model="deepseek-reasoner"
```

2. 设置提示词

提示词的设计对大模型的最终表现有显著影响。

一般而言，role:system，后面 content 字段填写模型的人设，例如，当你想让大模型帮你翻译文章，可以说：“你是一个专业的翻译专家”；当你想让模型回复变得友善，可以说：“你是一个友善的人”等。

role:user 后面的 content 字段填写要模型做的事情。

```
{"role": "system", "content": "你是一个有用的助手"},
{"role": "user", "content": "你好"}
```

为展示这种差异，阅读并运行 api_examples/prompt_comparison_demo.py 代码

表 1 运行结果总结

组别	对比	System 人设	输出特征	输出示例
组 1：翻译风格	对比 1	专业翻译专家（准确、正式、书面）	用词偏正式、客观中性	This mobile phone has excellent battery life...
	对比 2	社交媒体博主（口语化、自然、活泼）	词汇更轻松，语气更日常	This phone has amazing battery life...
	对比 3	技术评测编辑（简洁、评价语气）	简洁且保留评测语感	This phone has excellent battery life... just average.
组 2：客服回复	对比 1	友善同理客服专员	强同理心、安抚情绪、主动跟进	“非常理解您…非常抱歉…我会立即跟进处理…”
	对比 2	流程化售后经理（专业、条款感）	结构规范、流程清晰、偏制度语言	“根据服务条款第 3.2 条…请提供订单号及照片…”
	对比 3	高效客服机器人（50 字内）	极简回复、信息密度高	“非常抱歉…立即反馈物流和包装问题…”
组 3：信息提取	对比 1	信息抽取助手(自然语言总结)	一句话摘要，便于人读	“昨晚 8 点…今午收货发现外壳划痕…”
	对比 2	数据标注员（只输出 JSON）	结构化、字段固定，便于机器处理	{"time":"昨晚 8 点","location":"滨湖区门店",...}
	对比 3	运营分析师（总结+3 建议）	先归纳问题，再给可执行动作	“总结：… 可执行建议：1…2…3…”

补充：API 调用所需时间和多线程调用

正如在网页版问大模型问题，会需要一段时间模型才会返回结果。使用 API 调用模型同样会有速率限制。

面对大规模的数据，若逐一调用大模型，会耗费大量时间。为此，可以采用多线程（Multithreading）或异步调用方式，同时发送多个请求，在遵守速率限制的前提下大幅提升数据处理的吞吐量。实践中需注意合理控制并发数，避免触发平台

的限流机制。

相关展示代码在：api_examples/api_timing_multithread_demo.py【非必学】

Note：

了解有多线程并发调用这件事即可。在写调用大模型 API 代码的时候只需要关注主要执行逻辑，等大规模调用速度慢的时候，把原本的代码输入给大模型，prompt 写：“把这个代码改为多线程并发调用的代码，目前执行速度慢。”

大模型就会把改成并发调用的代码返回。

实践

实践案例：从用户评论中提取关键信息

任务背景：

线上平台有大量用户发表的产品评论，比如，在某汽车平台上，用户购买汽车后会进行购后评价。评价中蕴含着一些对企业而言有价值的信息，但信息往往比较稀疏。会有许多用户语气词，无需关注内容等噪音。

这对于企业分析用户需求是不利的。为此，需要从用户评论中提取出企业关注的内容。

例如，企业想知道线上用户都在讨论汽车的什么产品属性。

任务分析

- **Q1:人工如何提取出用户评论中提及的产品属性?**

人可以阅读每一条评论，手动提取每条评论中提及的产品属性。

- **Q2:大模型可以提取出用户评论中提及的产品属性吗?**

为了验证大模型的能力，我们可以先使用网页版的大模型（如 DeepSeek 网页聊天界面）**进行测试**。做法很简单：将一条用户评论输入对话框，询问模型“用户

提到了哪些车辆属性”。结果见图 3 网页版大模型回答结果分析

从结果来看，大模型能够准确理解用户表达的内容，并正确识别出评论中涉及的产品属性。

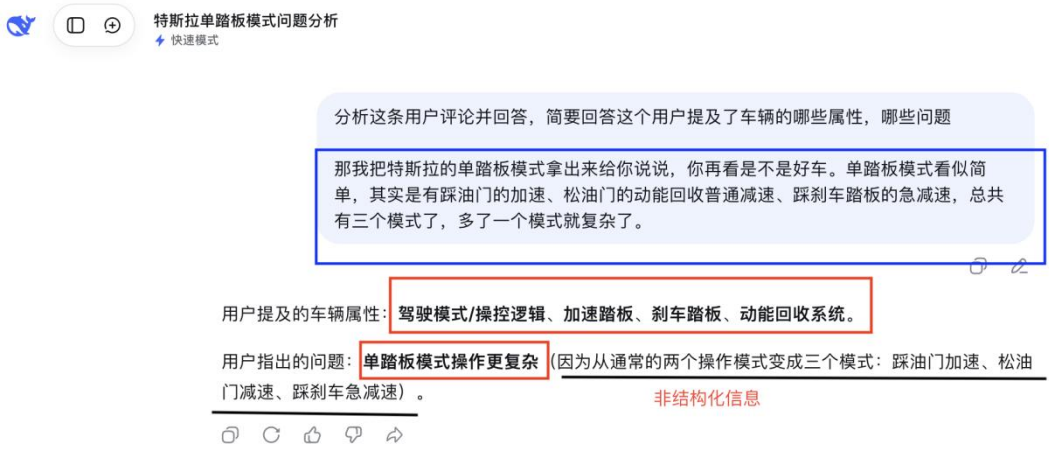


图 3 网页版大模型回答结果分析

补充分析：模型返回的内容中，除了我们想要的属性（如“P 档”“导航”），还有很多额外的描述性文字，比如“用户提及的车辆属性为……”“用户担心的是……”。

如果我们希望把这些结果存储到数据库中，或者进一步做统计分析，这些“多余”的文字反而会成为噪音。其实我们只关注红框里的信息，其他非结构化信息并不想存下来。那有没有方式，能让模型只回答我们想要的信息呢？

- **Q3:从网页版到批量处理——为什么需要 API?**

网页版虽然能验证可行性，但如果我们有几百上千条评论，不可能一条一条手动复制粘贴、再复制结果出来。

这时候，大模型 API 的优势就体现出来了：**每次调用只需要替换评论内容；可以用代码自动遍历所有评论；批量得到结果，并自动存储。**

模仿使用网页版大模型的做法，我们调用 api 批量处理 600 条数据 dataset，代码见 [api_examples/analysis_cases/process_car_review_data.py]

其中关注的部分为

```
# ===========关于调用大模型 API 主要代码区域===========
MODEL_NAME = "deepseek-chat"
SYSTEM_PROMPT = "你是一个用户评论分析助手，需要回答这个用户提及了车辆的哪些属性"
USER_PROMPT_TEMPLATE = "用户评论为：{content}"

def create_client() -> OpenAI:
    api_key = os.environ.get("DEEPSEEK_API_KEY")   #这里替换为 api_key 实际值
```

```
    return OpenAI(api_key=api_key, base_url="https://api.deepseek.com")

def call_model(client: OpenAI, user_text: str) -> str:
    user_message = USER_PROMPT_TEMPLATE.format(content=user_text)
    response = client.chat.completions.create(
        model=MODEL_NAME,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": user_message},
        ],
        stream=False,
    )
    return (response.choices[0].message.content or "").strip()
# ==================================================
```

可以看到这个代码就是 API 调用示例的组合。

遍历我们的用户评论数据，每次传给 USER_PROMPT_TEMPLATE 的 content 内容就是一条用户评论，这样批量获取结果

```
API 请求:  2%|                                                    | 14/600 [00:15<12:56,  1.32s/条
```

表 2 大模型 API 调用结果示例

content	model_response
也许吧 可螺丝压根不往里走….	根据用户评论，用户提及了车辆的“螺丝”这一属性，具体涉及螺丝安装过程中的问题（“螺丝压根不往里走”），可能反映车辆装配质量或部件匹配度相关的问题。
挂 p 档才可以用浏览器，那怎么驾驶时候看导航呢?	用户提及的车辆属性为：**P 档（驻车档）** 和 **浏览器**，以及 **导航功能**。其中，P 档是车辆的档位属性，浏览器和导航是车载系统功能属性。用户担忧的是 P 档限制下导航的使用体验。
评论之前看我说的 我哥们安的牌照 他是正规检车线的工作人员 每天拆装牌照很正常 我在 xpel 车衣授权店工作过，拆牌照再改色贴车衣属于常规操作 我们两个人都不是草包的情况下，前七个螺丝扣仅用了不到两分钟 最后这个螺丝反复找角度用了十分钟 最后滑扣 再瞎评论别找事 这是螺丝栓的品控问题	根据用户评论，提及的车辆属性如下： 1. **螺丝/螺丝栓的品控问题**：用户明确提到“这是螺丝栓的品控问题”，认为车辆牌照安装过程中最后螺丝反复调整角度仍滑扣，属于部件质量或品控缺陷。 此外，评论中还间接涉及： - **车牌安装相关部件**：如“螺丝扣”的匹配性和可操作性（用户提到前七个螺丝扣安装顺利，最后一个安装困难）。 用户主要聚焦于车辆牌照安装配件的质量属性。

同网页版调用一致，模型返回了很多嘈杂的，非结构化信息，虽然大模型可以理解用户提到的属性，但有太多回复性话语。

- **Q4:大模型如何只回复我们想要的内容?**

问题的关键其实在于我们怎么给模型下指令。

大模型非常依赖提示词（Prompt）的设计。如果我们希望它“只输出属性，不输出解释”，就需要明确告诉它这一点。

于是，我们修改系统提示词（SYSTEM_PROMPT）如下：

```
SYSTEM_PROMPT =  """你是一个用户评论分析助手，需要回答这个用户提及了车辆的哪些属性
仅需要回复车辆属性，不需要回复其他内容
多个属性用逗号分隔
回复示例：动力性能,外观设计,内饰配置,安全性能,操控性能,舒适性,性价比,售后服务,品牌影响力,其他
请根据用户评论，回答用户提及了车辆的哪些属性
"""
```

我们把这个新的提示词复制到源代码对应位置，重新运行一遍结果，可以发现

表 3 新版提示词大模型 API 调用结果示例

content	model_response
也许吧 可螺丝压根不往里走….	安装,螺丝
挂 p 档才可以用浏览器，那怎么驾驶时候看导航呢?	导航系统
评论之前看我说的 我哥们安的牌照 他是正规检车线的工作人员 每天拆装牌照很正常 我在 xpel 车衣授权店工作过，拆牌照再改色贴车衣属于常规操作 我们两个人都不是草包的情况下，前七个螺丝扣仅用了不到两分钟 最后这个螺丝反复找角度用了十分钟 最后滑扣 再瞎评论别找事 这是螺丝栓的品控问题	螺丝扣,螺丝栓

这样的结果在下游更加好分析。

总结

从本案例中，可以提炼出一条清晰的问题解决思考链。**第一步是明确目标与现状：**企业希望从大量嘈杂的用户评论中提取出用户讨论的产品属性，而原始评论包含大量语气词和无关信息。**第二步是人工可行性验证：**思考人如何完成这项任务——人可以逐条阅读并准确提取属性，这说明任务本身是“可被理解的”，适合自动化。**第三步是最小工具验证：**用网页版大模型直接测试，发现模型能正确识别属性，证明技术可行，但也暴露出输出中包含了大量解释性文字，不利于存储和下游分析。

第四步是定位根因：问题的关键不是模型能力不足，而是我们没有对输出格式施

加任何约束。通过控制输入来优化输出：修改系统提示词，明确限定只输出属性词、用逗号分隔、并给出示例。这一调整让输出从“正确的废话”变为“干净的结构化数据”，可直接入库分析。

最终，将优化后的提示词与 API 批量调用结合，实现了从人工一条条处理到自动化大规模提取的跃升。

学有余力，其他提示词修改方式、输出格式见后续章节提示词写法

课堂作业：某领域新闻分类

问题背景：

在真实业务场景中，行业资讯通常来自多个线上渠道（新闻网站、公众号、行业平台、报告转载等），每天新增内容很多。

如果只是“堆在一起”，读者很难快速定位自己关心的信息，编辑和运营也难以进行专题整理、热点跟踪与知识沉淀。

因此，需要先对资讯做自动分类。

分类后可以支持：按主题阅读（例如政策、企业动态、技术前沿）;快速筛选与订阅（只看自己关注的类别）;构建专题知识库（同类内容聚合）;做后续的数据分析（热点变化、类别趋势、信息覆盖度）

本次课堂作业模拟这一真实流程：

你将使用大模型对新闻标题进行自动分类，并与人工标注的真实类别对比，评估模型在实际信息整理场景中的可用性。

作业内容

打开代码 [api_examples/analysis_cases/news_classification_task.py](), 需要修改代码对应片段

```
MODEL_NAME = "deepseek-chat"
# ====================== 在此修改系统提示词 ======================
# 类别有6个，分别是：政策追踪，国外科技前沿，典型案例，院士专家观点，电力企业动态，国内科技前沿
SYSTEM_PROMPT = "你是一个文本分类助手。要对以下标题进行分类xxxx【这只是一个示例，需要替换】"

USER_PROMPT_TEMPLATE = (
"标题：{title}\n"
)
# ====================== 在此修改系统提示词 ======================
```

修改好后进行运行，代码会显示出当前方案分类的准确率，并且大模型返回的分类结果会存在 *OUTPUT_XLSX = "data/news_data_result.xlsx"*中

作业交付

基础目标：分类准确率 70%以上。

同步要进行：

- 错误分析（必须）：根据大模型返回的分类结果 *news_data_result.xlsx*，分析没有成功被分类的样本【当前判断样本是否被正确分类需要返回的字符串完全和类别一致】

  典型错分样本分析若干条（标题、真实类别、预测类别）
- 改进过程（建议至少 2 轮）：

  第 1 版提示词与准确率

  改进点与第 2 版准确率
- 结论：

  是否达到要求

  有哪些可继续优化方向

、

大模型微调

该章节代码、数据对应位置：HFUT-DA-practicum/sft，使用的平台为火山引擎模型精调

微调介绍和数据格式

微调（Fine-Tuning）是指在预训练大模型的基础上，使用特定任务的数据集进行进一步训练，使模型更好地适应具体应用场景。

有监督微调（Supervised Fine-Tuning, SFT） 是其中最常见的一种方式：通过包含示例输入与标准输出的数据集对模型进行训练，从而提升模型在特定任务上的表现与执行效率。

简单来说：你给模型看一批“问题-正确答案”的例子，它学会按你期望的方式来回答。

> *有监督微调（Supervised Fine-Tuning, SFT）通过包含示例输入与标准输出的数据集对模型进行微调，可提升模型在特定任务上的效果与执行效率。*
>
> *在以下几种情况，您可以考虑使用有监督微调（SFT）：*
>
> ***尽管您已尽力优化提示词工程（Prompt Engineering），但模型效果仍不理想，或提示词本身变得过于复杂。***
>
> ***您对模型输出有特定的格式要求，但它有时无法稳定地满足要求。***
>
> ***您希望精简提示词，以加快线上推理速度。***
>
> *----《火山方舟文档指南》*

总的来说，微调指的是，通过给定的模型输入和输出示例，让模型学会相应的回答。

微调效果很大程度上取决于数据质量。以下是常见的**数据格式**：

1. 对话格式

```
[
  {
    "messages": [
      {"role": "system", "content": "你是一个专业的产品经理助手。"},
      {"role": "user", "content": "如何撰写产品需求文档？"},
      {"role": "assistant", "content": "PRD 通常包含以下章节：1. 背景与目标 2. 功能列表 3. 交互流程..."}
    ]
```

```
  }
]
```

2. 指令格式

```
{
  "instruction": "将以下句子翻译成英文",
  "input": "今天天气很好",
  "output": "The weather is nice today."
}
```

常见微调方法对比

表 4 微调方式简要对比

方法	简介	适用场景
全量微调	更新模型全部参数	数据充足、有较强算力
LoRA	仅训练低秩适配器	资源有限、快速实验
QLoRA	量化 ＋LoRA	单卡可微调大模型

实践

实践案例：微调一个用户评论信息抽取模型

任务背景：

线上平台有大量用户发表的产品评论，比如，在某汽车平台上，用户购买汽车后会进行购后评价。评价中蕴含着一些对企业而言有价值的信息，但信息往往比较稀疏。会有许多用户语气词，无需关注内容等噪音。

在前一节的实践中，我们已经成功通过优化提示词，让大模型从嘈杂的用户评论中提取出干净、逗号分隔的产品属性词，解决了模型“说废话”的问题。

然而，在实际业务落地中，这种“靠提示词约束格式”的方法仍然面临几个难以回避的挑战

- 每次调用 API，我们都需要在 SYSTEM_PROMPT 中携带大段格式约束指令（示例中的那段几十个字的“仅回复属性、用逗号分隔”的提示词）。对于高频调用场景，这会额外消耗输入 Tokens，增加成本。
- 大模型本质上是一个对话生成模型，提示词只是“建议”，并非硬性约束。在复杂评论（如讽刺、长篇幅情绪话）中，模型仍可能“忍不住”给出解释性文字，导致下游解析异常。我们无法 100% 保证输出格式稳定。
- 有其他小模型部署需求

问题分析和数据准备

有没有一种方法，能让模型内化我们的输出规则，从而不再依赖每次携带的提示词？

答案是微调（Fine-tuning）。

微调的核心思想是：构造一批“输入-期望输出”的样本，让模型在这些样本上进行额外的参数更新。训练完成后，模型在面对新的用户评论时，会自然地按照训练时学到的格式输出，而无需每轮重复指令。

本章微调实验所使用的数据，来源于上一节提示词工程处理后的结果。

原始的 Excel 表格中，两列核心字段为：

表 5 数据构造核心字段

原始字段	含义	示例
content	用户原始评论	“挂 p 档才可以用浏览器，那怎么驾驶时候看导航呢？”
model_response	提示词工程处理后的属性词	“导航系统”

为了适配大模型微调的标准数据格式（通常为 JSONL，每条数据包含 messages 数组），我们需要将上述两列构造为以下结构：

```
{
  "messages": [
    {
      "role": "user",
      "content": "提取评论中的属性，评论为：挂 p 档才可以用浏览器，那怎么驾驶时候看导航呢？"
    },
    {
      "role": "assistant",
      "content": "导航系统"
    }
  ]
}
```

我们使用代码 excel_to_sft_jsonl.py 进行转换，此处给出已经转换好的数据 car_review_sft_dataset.jsonl

微调过程

在本实践课程中，我们使用火山方舟平台进行模型微调。

火山引擎模型精调

图 4 进入平台：点击大模型 -> 模型开发 ->火山方舟大模型服务平台

创建数据集

图 5 火山方舟创建数据集：点击数据集

创建数据集，上传文件 car_review_sft_dataset.jsonl

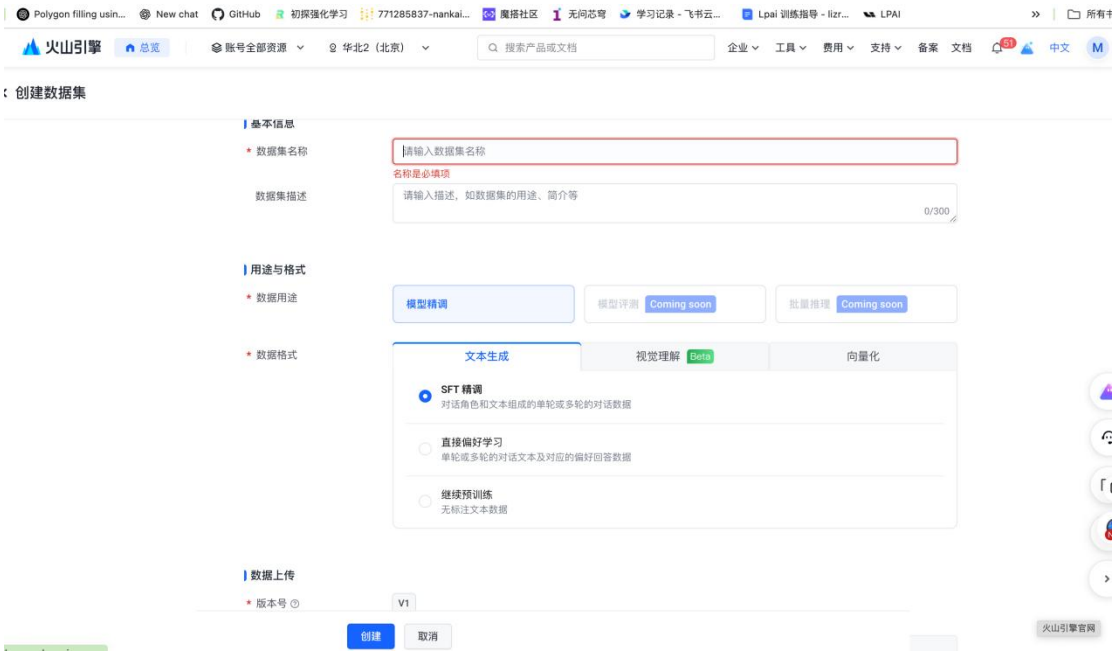


图 6 上传数据集

模型微调

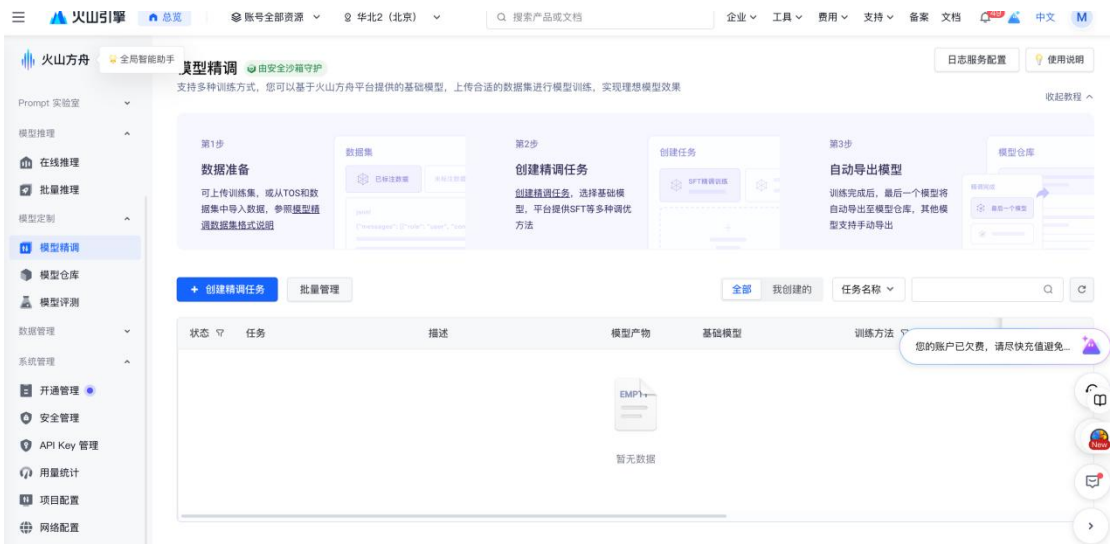


图 7 创建精调任务，点击模型精调->创建精调任务

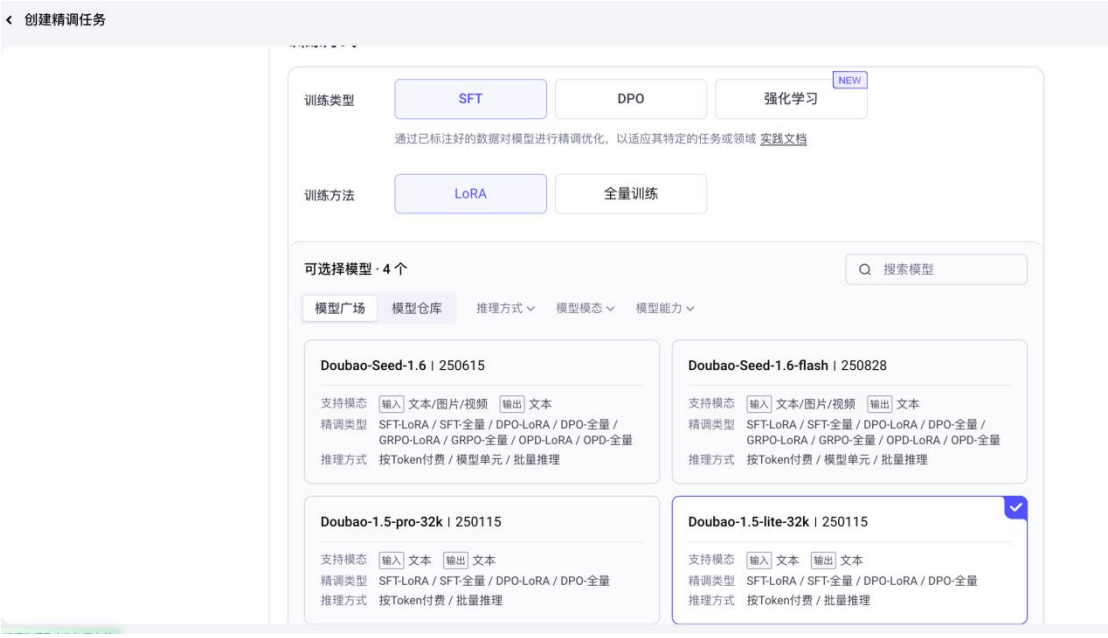


图 8 选择训练方式：SFT、LoRa,模型选择 doubao-1.5-lite(相对轻量)

图 9 数据集选择之前上传过的数据集

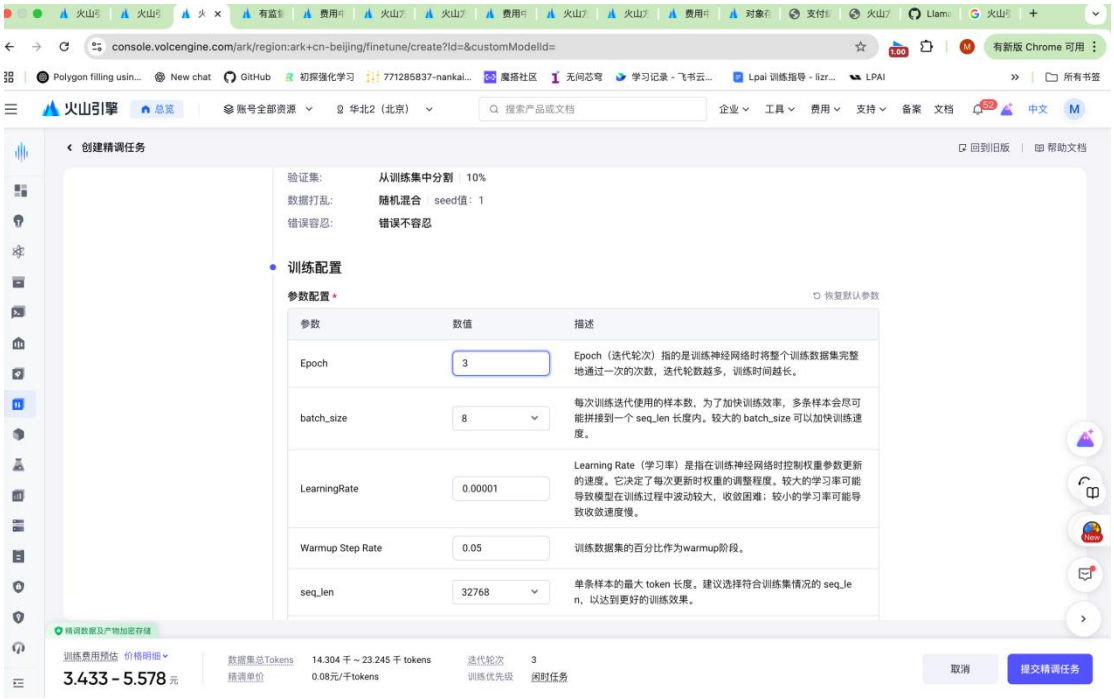


图 10 训练参数选择 epoch=3（测试 epoch=1，发现效果不好，于是增加了次数）

微调结果

等待微调完成后，点击立即体验

图 11 微调模型结束

我们可以比较微调前后的效果。

首先，点击立即体验，体验微调后的我们的模型

图 12 体验微调后模型

体验原模型

图 13 点击左上角，选择模型->语言->更多模型->doubao1.5-lite-32k

同样的提示词，看效果。

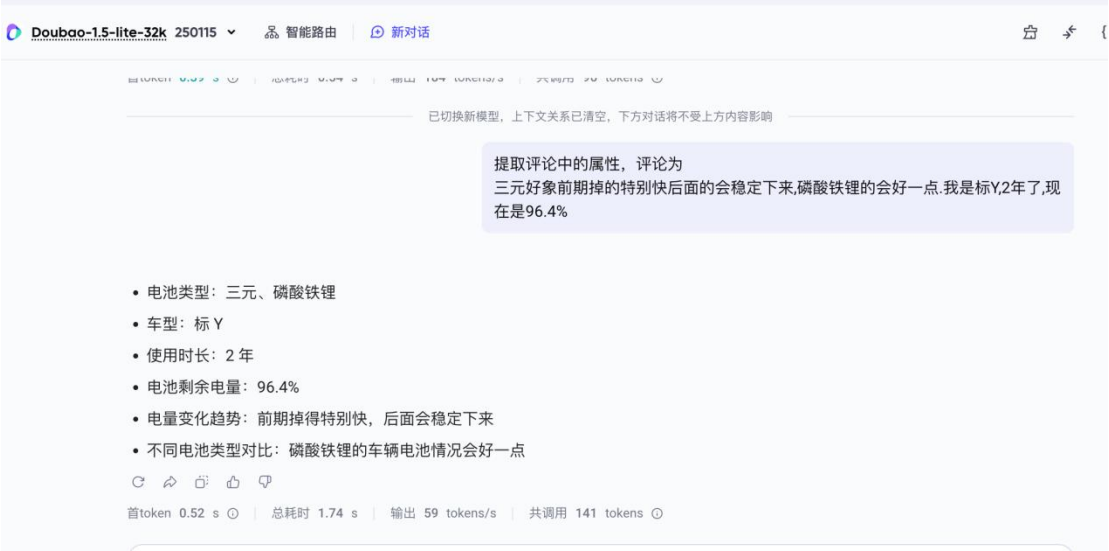


图 14 原模型会回答冗余的内容

可以发现微调后的模型会按照我们指定的方式进行回复。而原模型的回复格式不是我们所期望的。

课堂作业：微调模型展示

作业内容

完成实践案例的步骤，比较原模型和微调后的模型效果

作业交付

1. 微调后的模型页面展示
2. 对于特定场景，若要微调，微调的数据格式是怎样的，理想中各个字段应该是什么，可以从哪里获得这些数据。

e.g. 我想做一个文章精简助手，给一篇文章得到精简后的内容。如果要微调大模型，数据准备阶段该如何做。

Vibe coding 实践

“Vibe Coding”是一种近年来兴起的编程方式，指开发者不再逐行手写代码，而是通过自然语言与 AI 对话，让 AI 理解需求并生成、修改、调试代码。开发者更像一个“导演”或“产品经理”，把握大方向、理解业务逻辑、判断 AI 输出是否合理，而具体的代码实现交给 AI 完成。

该部分示例代码在 HFUT-DA-practicum/vibe_coding_case

AI IDE 介绍与下载

表 6 常见的 AI IDE 与 AI 编程工具

工具名称	类型	特点
Cursor	AI IDE	基于 VS Code 深度改造，AI 原生支持，支持多文件编辑、代码补全、对话式编程
GitHub Copilot + VS Code	插件	最早普及的 AI 编程工具，代码补全能力强，与 GitHub 深度集成
Claude Code	命令行工具	Anthropic 官方出品，在终端中与 Claude 对话，支持代码理解、修改、提交等操作
Codex	底层模型/API	OpenAI 推出的代码生成模型，GitHub Copilot 早期就基于 Codex，也可通过 API 直接调用
Trae	AI IDE	新兴 AI IDE，界面简洁，与国内大模型适配较好

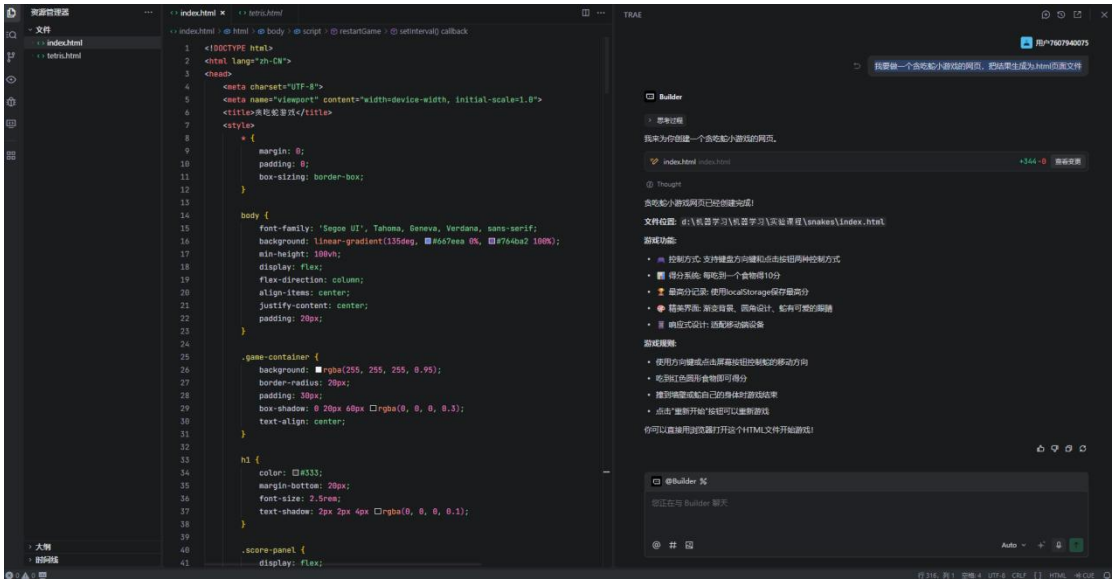
在本次课程中，我们将使用 Trae 作为 AI IDE 实践工具。

相关下载使用教程为：TRAE 下载与使用

能力初探

我要做一个贪吃蛇小游戏的网页，把结果生成为.html 页面文件

在 Trae 的右侧输入以上提示词。让其自动生成一个小游戏。

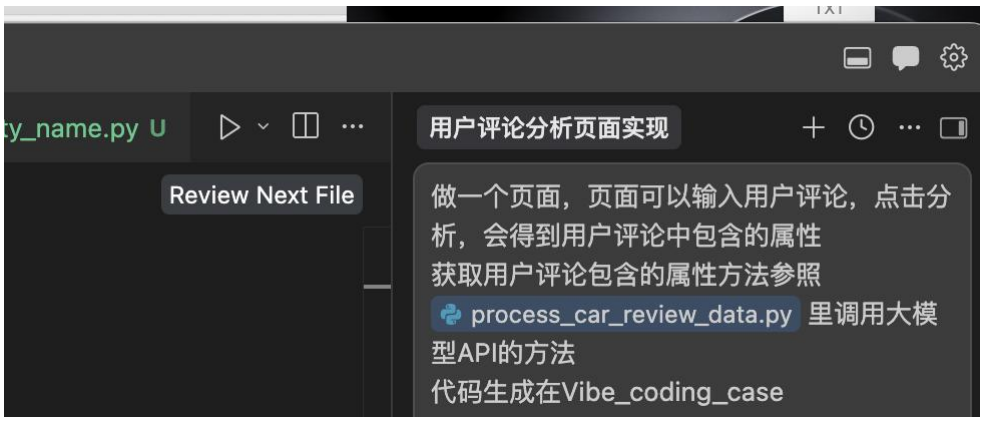


运行成功之后，在文件夹中双击.html 文件，便可在网页执行如下界面：

实践

实践案例：简易的用户评论分析器

通过以上的实践，我们已经知道如何调用 API 从用户评论中获取关键信息。现在，我想做一个页面，用户输入自己的评论，页面便会展示提取出来的属性。在明确需求后，利用 AI IDE，我们来完成这个工作。



面对简单任务，一段如上指令即可完成。接着 AI IDE 内嵌的模型写好了代码并给出了运行方式。

运行后得到一个简易的用户评论分析器

图 15 页面与功能演示

课堂作业：数据分析相关的小工具

作业内容

利用 AI IDE 做一些数据分析相关的小工具。

例如能否传一个 excel 表，自动分析得到一些数值类信息。

或能否传入一条新闻标题，分类其属于哪个领域。

作业交付

可运行的数据分析工具代码。

拓展资料

这一部分作为补充章节。

一方面，对实践教程的 API 调用、大模型微调相关知识进行进一步补充。Section 提示词写法、模型微调 附上一些可进一步了解的资料。

另一方面，提供部分大模型在科研、产业界应用上的实际做法，这里的科研聚焦于管理学中获取用户需求等方向、产业界应用聚焦于大模型应用岗以及部分非研发岗可能可以结合大模型做的工作。

这里的介绍远非系统知识和方案，仅做抛砖引玉。

建议为：若日后愿攻读管理学相关研究生，可了解 Section 管理学科研领域中的大模型，或许在研究某个问题的某个小版块，会想到有种利用大模型的方法可以解决；若进入研发岗工作，阅读该拓展资料，更多探索的知识可以通过各种开源资料等深入学习。

提示词写法

提示词基本要素

一个完整的提示词通常由以下七个要素构成：

基本公式： 角色 + 指令 + 上下文 + 问题 + 约束 + 例子 + 输出格式

表 7 提示词要素

要素	作用	关键价值
1. 角色	为 AI 赋予特定身份（如“资深环境学家”“侦探小说作家”）	让输出更专业、个性化，提升回答准确性
2. 指令	明确告知 AI 需要完成的任务	核心驱动要素，决定 AI 做什么
3. 上下文	提供与任务相关的背景信息	帮助 AI 理解具体情境，使输出更精准
4. 问题/输入	需要 AI 执行的具体任务或回答的具体问题	当指令已描述任务时，此部分可合并
5. 约束	设定输出范围或限制条件	有效控制 AI 的回答边界
6. 输出格式	指定答案的组织形式（如分点、表格、JSON 等）	确保结果符合使用需求
7. 例子	提供 1-2 个参考示例	显著提升输出的准确度

并非每次提问都必须包含所有要素。简单问题可能只需要“指令+问题”，复杂

任务则建议组合更多要素以获得理想效果。

提示词优化

- Bad Case 分析

Bad Case 是指模型在当前提示词下输出不符合预期的样本。

Bad Case 分析流程为

测试一批数据 → 筛选输出异常的样本 → 分析原因 → 修改提示词 → 重新测试

- Few-shot 学习

Few-Shot 是指在提示词中提供几个输入-输出示例，让模型通过类比学习任务的输出格式和逻辑。

大模型本质上是“下一个词预测”模型。当我们在提示词中给出几个高质量示例时，模型会：自动推断输入和输出之间的映射规律；模仿示例中的输出格式和风格；减少对任务描述的依赖（示例本身就是最强约束）

- 优化输出格式

在前面的章节中，我们通过提示词工程让模型输出逗号分隔的属性词（如 动力性能,智能座舱,能耗经济性）。这种格式虽然比原始的“解释性废话”干净很多，但在真实业务场景中仍然不够用。

为什么？因为不同的下游任务对数据格式有不同的要求，例如：

统计分析：需要知道每个属性被提及的次数

情感分析：需要区分用户是正面还是负面评价

数据库存储：需要结构化的 JSON 格式

可视化展示：需要带有属性名称的键值对

这一部分将介绍如何针对不同业务目的，设计最优的输出格式，并给出完整的提示词模板

先看一个例子。原始评论：

“这车动力很强劲，但油耗太高了，另外导航系统反应很慢”

逗号分隔格式（基础版）：

动力性能,能耗经济性,智能座舱

问题：下游无法知道用户对每个属性的态度（正面还是负面），也无法知道具体的描述细节。

优化后的 JSON 格式：

{ "动力性能": {"情绪": "正面", "关键词": ["强劲"]}, "能耗经济性": {"情绪": "负面", "关键词": ["油耗高"]}, "智能座舱": {"情绪": "负面", "关键词": ["导航", "反应慢"]}}

这种格式可以直接存入数据库，也可以直接用于可视化分析。

提示词示例

```
你是一个用户评论分析助手。请从用户评论中提取用户提到的车辆属性及相关详情，以 JSON 对象格式输出。
输出格式：
{
  "属性名称 1": {
    "情感": "正面/负面/中性",
    "关键词": ["词 1", "词 2"],
    "原文片段": "用户原话中相关的那句话"
  }
}
要求：
- 关键词从评论中提取，每个属性最多 3 个关键词
- 原文片段尽量保留原话，不超过 20 字
- 如果没有提到任何属性，输出空对象 {}
用户评论：{comment}
输出（仅输出 JSON，不要有其他内容）：
```

模型微调与推理

常见用于微调的框架

https://llamafactory.cn/llm-integration/overview.html

https://github.com/verl-project/verl

推理加速

vllm

在微调中，高质量数据是最重要的，除此之外，在 RL 中，合适的环境和 Reward 也尤为重要。

这一部分可通过实践逐步了解领会。

管理学科研领域中的大模型

大模型作为工具

在管理学研究中，大模型可以作为**实现研究目标的辅助工具**，应用于文本测量、数据标注、内容生成与研究流程自动化等环节。

这一部分有理论依据，不少论文在做“大模型能否完成某类任务”，这属于将大模型本身作为研究对象或方法有效性评价对象的研究范式，见 Section 大模型作为研究对象

> *e.g.*
>
> *从文本中提取心理结构对于研究态度、感知和特质的社交科学研究人员来说越来越重要了。我们的结果表明，大型语言模型在性能上可以匹敌或超过现有的监督方法，并且能够生成更加一致和公平的预测，而无需专门的自然语言处理专业知识或大量的标注数据。*
>
> ----From Lexicons to Large Language Models: A Holistic Evaluation of Psychometric Text Analysis in Social Science Research
>
> 《Information Systems Research》于 2026/4/22 出版的论文

但在这一部分，我们只关注论文中如何将大模型作为工具合理嵌入研究流程，并提高研究数据处理、变量构建和文本分析的效率。

大模型介入数据预处理阶段

在数据预处理阶段，不同类型的数据清洗任务适合采用不同的方法。这里给出两个场景示例：

场景 A	场景 B
我们通过问卷获取了一组数据，要筛选出可用的问卷数据。例如，对于作答时间明显过短、存在明显直线作答倾向、正反向题答案严重矛盾以及开放题填写无意义内容的问卷，将其判定为无效样本并予以剔除	我们为研究 A 主题，在线上通过关键词爬取了诸多数据，其中包含和 A 主题无关的噪声数据，只想得到谈论 A 主题相关的文本。

对于具有明确数值标准或显式判断规则的任务，可以优先采用规则式方法进行处理。例如，在场景 A 问卷数据清洗中，作答时间明显过短、连续选择同一选项、缺失值比例过高等问题，通常可以通过设定阈值或编写代码规则直接识别和剔除。这类任务具有较强的结构化特征，判断标准相对明确，因此不一定需要引入大模

型。

但对于判断标准较为模糊、依赖语义理解的清洗任务，大模型则具有更高的应用价值。例如，在问卷开放题中，研究者需要判断受访者是否填写了无意义内容、回答是否与题目相关，或者正反向题之间是否存在较复杂的逻辑矛盾。此时，仅依赖关键词或固定规则可能难以准确识别，大模型可以通过语义理解辅助完成无效样本识别。

类似地，场景 B 中，传统方法通常依赖关键词匹配、反向关键词排除或人工筛选，但这些方法容易受到表达方式多样、语义隐含和语境差异的影响。相比之下，大模型可以根据文本整体语义判断其是否真正讨论研究主题，从而提高主题相关文本筛选的准确性。而当我们要删除过长或过短的评论，也可进行直接代码筛选。

总的来说，大模型更适合处理那些**非结构化、语义性强、难以通过固定阈值或显式规则穷尽的筛选任务**。在实际研究中，更合理的做法是**将规则式方法与大模型方法结合使用**：先用代码规则处理长度、时间、缺失值等客观指标，再用大模型处理主题相关性、语义有效性和开放文本质量等判断任务。这样既能提高处理效率，也能增强数据清洗过程的解释性和可复现性。

Note: 大模型可以作为数据预处理工具介入研究流程；具体采用规则筛选、关键词筛选还是大模型筛选，应根据数据类型和清洗任务决定；在论文中只需要简要说明数据筛选与预处理过程，不必展开过多技术细节。

大模型作为框架中的一环

本质上，大模型属于深度学习模型，是在大规模数据和大规模参数基础上训练得到的神经网络模型。

以往我们为了研究某个管理学问题，借助神经网络模型，这些模型根据任务适配性可以选择图神经网络，选择 LSTM 或 CNN 或其他，现在多了一个选择是大模型。

因此，在管理学研究中，模型可以在任务适配和有效性验证的基础上，作为新的模型模块嵌入其中。并且由于大模型的特性，还可以创造新的嵌入方式。这里给出几个示例。

研究主题：虚假新闻预测

论文：DELL: Generating Reactions and Explanations for LLM-Based Misinformation Detection

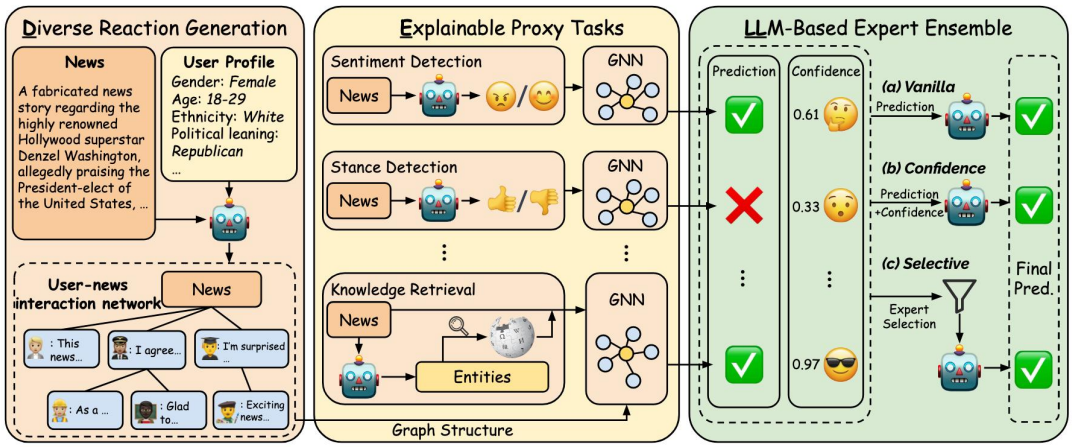


Figure : Overview of DELL. We first employ LLMs to generate news reactions from diverse perspectives and form user-news interaction networks. We then design six explainable proxy tasks to refine the feature embeddings with LLM-generated explanations. We finally propose three LLM-based strategies to selectively merge the predictions of task-specific experts and enhance calibration.

DELL 概述。我们首先使用 LLM 从不同的角度生成新闻反应，并形成用户新闻互动网络。然后，我们设计了六个可解释的代理任务，用 LLM 生成的解释来细化特征嵌入。最后，我们提出了三种基于 LLM 的策略，有选择地合并特定任务专家的预测并加强校准。图 1:DELL 概述。我们首先使用 LLM 从不同的角度生成新闻反应，并形成用户新闻互动网络。然后，我们设计了六个可解释的代理任务，用 LLM 生成的解释来细化特征嵌入。最后，我们提出了三种基于 LLM 的策略，有选择地合并特定任务专家的预测并加强校准。

分析：一方面，大模型在这篇论文里当了一个分类器，GNN 输出分类结果和置信度，再外加了一层 LLM 分类（可以理解为外接了一个神经网络）；另一方面，LLM 发挥了自己的“专长”，通过模拟用户，形成新闻互动网络，增加了输入的信息量（从某种角度，也可以也理解为外接了一个神经网络）

研究主题：财务管理/会计（信息披露）

论文：DELL: Generating Reactions and Explanations for LLM-Based Misinformation Detection

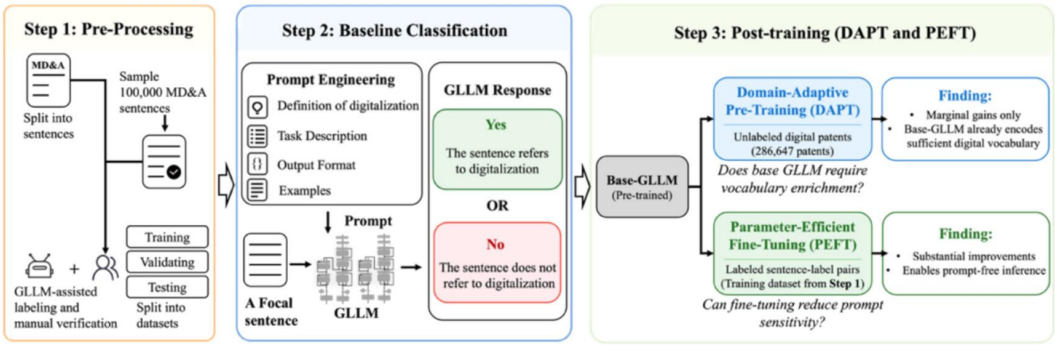


Figure : Three-step methodology for GLLM-based digital content classification

基于 GLLM 的数字内容分类三步法

分析：这篇论文开发了一个 GLLM 框架，用于对年度报告的管理讨论和分析（MD&a）部分中的数字相关内容进行分类。之前数据预处理、特征处理、训练分类器变成了围绕 LLM 微调的框架。

结果评判

LLM-as-a-Judge 适用于人类标注成本高、评估标准可明确文本化、或需要大规模批量化评估的场景。

例如在以下文献中，作者想证明自己提出的方法生成的摘要比别的方法好，用 LLM 对各种方法生成的结果进行打分，得到自己分高。

Judge / Config	Selene-1-Mini-Llama-3.1-8B					Llama-3.3-70B-Instruct				
	COH	CON	FLU	REL	Avg	COH	CON	FLU	REL	Avg
Llama 3.1	3.79	4.12	3.54	3.86	3.82	4.26	4.34	3.92	4.29	4.20
Gemma	3.77	4.13	3.62	**4.04**	**3.89**	**4.41**	4.39	**4.12**	**4.52**	**4.36**
Qwen	**3.88**	4.12	3.63	3.96	**3.89**	4.29	4.37	3.98	4.41	4.26
0-Shot	3.86	**4.14**	**3.66**	3.86	3.88	4.28	**4.40**	4.00	4.36	4.26
Reference	3.47	3.91	3.31	3.52	3.55	3.89	3.92	3.60	3.86	3.81
QUARTZ	**3.89**	**4.18**	**3.66**	3.87	**3.90**	**4.47**	**4.41**	3.96	**4.53**	**4.34**

Table 6: LLM-as-judge evaluation scores for summaries using two models. COH, CON, FLU, REL, and Avg denote coherence, consistency, fluency, relevance, and average score, respectively.

来源：Mohamed Imed Eddine Ghebriout, Gaël Guibon, Ivan Lerner, Emmanuel Vincent. QUARTZ: QAbased Unsupervised Abstractive Refinement for Task-oriented Dialogue Summarization. EMNLP 2025 : The 2025 Conference on Empirical Methods in Natural Language Processing, Nov 2025, Suzhou, China. hal-05300943

大模型作为研究对象

这类研究关注的核心问题是“大模型在特定任务中是否具有接近、替代或补充人类判断的能力，以及这种能力的边界在哪里”。因此，此类研究通常需要设置外部参照基准，例如人类调查结果、人工标注结果、传统机器学习模型、传统数字渠道或真实市场行为数据，通过比较大模型输出与基准结果之间的一致性、稳定性和预测效度，判断大模型在具体管理情境中的适用范围。

论文名称	来源	研究对象/任务	核心观点
Frontiers: Determining the Validity of Large Language Models for	Marketing Science	检验 LLM 是否可在市场研究中替代或补充人类参	LLM 生成结果与人类调查结果高度接近；在人类与 LLM 生成数据之间，一致

Automated Perceptual Analysis		与者进行感知分析。	率超过 75%，并适用于品牌相似性和产品属性评分等感知分析任务。
Frontiers: ChatGPT Referrals to E-Commerce Websites: How Do LLMs Compare Against Traditional Channels?	Marketing Science, Articles in Advance	将 ChatGPT/LLM 自然引流作为电商渠道，比较其与传统数字渠道的商业表现。	基于 973 个电商网站、12 个月一方数据，研究发现 ChatGPT 自然引流的转化率和每次访问收入高于 paid social，但低于多数传统渠道；复杂产品类别中表现更强。
The Use of LLMs to Annotate Data in Management Research: Foundational Guidelines and Warnings	Strategic Management Journal	讨论 LLM 在管理研究数据标注和文本分类中的实施规范、有效性与风险。	LLM 可在较低成本下达到或超过传统方法表现，但提示词变化会显著影响结果及后续实证分析；因此需要敏感性分析、人工验证和透明报告。
Experimental Evidence on the Productivity Effects of Generative Artificial Intelligence	Science	通过预注册在线实验考察 ChatGPT 对中等专业写作任务生产率和质量的影响。	ChatGPT 显著提升专业写作任务完成效率和质量，并降低能力差异；但研究场景限定在可由 AI 辅助完成的写作任务。
Generative artificial intelligence and evaluating strategic decisions	strategic management journal	使用 LLM 评估战略决策	生成式人工智能的单一评估往往不一致或有偏见。然而，如果管理者在 LLM、提示或角色之间汇总了许多评估，结果表明，由此产生的评估往往与人类的评估相似.

其他

还有诸多其他方面研究，例如，研究大模型对社会舆论的影响，使用大模型对人的影响，组织行为与大模型，但这一块不太了解因此不过多赘述。

大模型面向特定领域的后训练

为提升模型在特定领域的能力，往往会采用对基座模型进行后训练。

1. 数据合成阶段

这一块可参考 SWE Bench 数据合成论文等。

2. SFT 阶段

SFT 的核心在于模仿。概念上很简单：你强制性地一步一步教模型学会一个答案。

对于 LLM 来说，SFT 的过程就是逐字学习理想答案。因此，损失函数就是输出层的交叉熵（cross-entropy）。

在 SFT 阶段，high quality data is all you need。

3. 强化学习阶段

这里主要介绍 PPO

核心目标: 希望找到一个策略使得智能体在环境中执行动作好获得长期奖励期望最大化。

PPO 通过引入裁剪机制来限制更新的幅度。包括 actor model, reference model, critic model，reward model，把语言模型当作 Actor，reference model 防止更新幅度大，critic model 估计状态的预期回报，reward model 近似人类打分偏好。

优化目标:

$$\mathbb{E}_t\left[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)A_t) - \beta \text{KL}[\pi_{\theta_{old}}(\cdot|s_t)||\pi_\theta(\cdot|s_t)]\right]$$

可以简单看作，优化每一步的 Advantage（对应 A_t），让其尽可能大

- **在 LLM 里面每一步的 Advantage 是什么?**

在 LLM 里，可以把一步步生成 token 看为“每一步”，advantage 是在描述生成的该 token 有多好。

由于 reward model 是预先训练的, 我们假设其能很好的衡量生成内容应当得到的奖励。自然想到，让 advantage = reward model 输出的奖励。

这里是可行的, 不过在实际训练中, reward model 通常是针对整个回答进行打分, 不能细致的打到每个 token 上。其实可以直接在每个 token 上加 reward model 针对整个回答的打分，或通过某种方法分配分值，但在工程实践中，只把 reward 分值加到最后一个 token。不论如何，我们可以认为，当我们有 reward，那 reward 可以作为一种 advantage。

这里有个疑问，好和坏是相对的，能否用相对值来衡量 advantage? 可以。

比如用均值等方式，将生成该 token 的 reward 减去某个基准值，就变成相对了。

在 PPO 中，引入一个 critic model，用来衡量预期收益 V_t 表示第 t 步预期的未来收益。

某一步的 token 的 advantage 定义为：选择生成该 token 的当下收益+未来收益比生成其他的内容收益多多少。

$$A_t = R_t + \gamma V_{t+1} - V_t$$

后面的式子便是 TD error，如果 V_t无偏，则A_t无偏。可惜，critic model 是模拟出来的，往往有偏。为了降低偏差，定义 A_t为未来所有步 TD error 的指数加权平均。这便是 GAE，也是 PPO 里真实计算 advantage 的公式。

$$A_t^{GAE} = \sum_{l=0} (\gamma\lambda)^l TD_{t+l}$$

在计算时，可以通过递推计算，已知 $A_t=0$(if t > length)，也已知 R_t和 V_t,可以从后往前递推

$$A_t^{GAE} = \gamma\lambda A_{t+1}^{GAE} + TD_t$$

在 deepspeed 代码里是这么写得

DeepSpeedExamples/applications/DeepSpeed-Chat/dschat/rlhf/ppo_trainer.py 第 308 行起

```
def get_advantages_and_returns(self, values, rewards, start):
    # Adopted from <https://github.com/CarperAI/trlx/blob/main/trlx/models/modeling_ppo.py#L134>
    lastgaelam = 0
    advantages_reversed = []
    length = rewards.size()[-1]
    for t in reversed(range(start, length)):
        nextvalues = values[:, t + 1] if t < length - 1 else 0.0
        delta = rewards[:, t] + self.gamma * nextvalues - values[:, t]
        lastgaelam = delta + self.gamma * self.lam * lastgaelam
        advantages_reversed.append(lastgaelam)
    advantages = torch.stack(advantages_reversed[::-1], dim=1)
    returns = advantages + values[:, start:]
    return advantages.detach(), returns
```

这里补充一下，该代码里面的 returns 为每步出发的折扣回报估计，前面说 GAE advantage 是在计算当前步能得到的回报与期望回报差值，那么 advantages + values[:, start:]就为当前步得到的回报估计。returns 在后续会用来更新 critic model，因为当策略（actor model）更新后，模型会倾向于生成高 advantage 的回答，那么 critic model 也要相应更新为对应策略的价值估计，和 returns 值应当接近。

在代码里，loss 是这么计算的

DeepSpeedExamples/applications/DeepSpeed-Chat/dschat/rlhf/ppo_trainer.py 第 292 行起

```
def critic_loss_fn(self, values, old_values, returns, mask):
    ## value loss
    values_clipped = torch.clamp(
        values,
```

```
            old_values - self.cliprange_value,
            old_values + self.cliprange_value,
        )
        if self.compute_fp32_loss:
            values = values.float()
            values_clipped = values_clipped.float()
        vf_loss1 = (values - returns)**2
        vf_loss2 = (values_clipped - returns)**2
        vf_loss = 0.5 * torch.sum(
            torch.max(vf_loss1, vf_loss2) * mask) / mask.sum()
        return vf_loss
```

回到 Advantages 计算，具体而言，value 是 critic 模型输出的，每个 token 位置对应值。

那我们刚刚提到 reward model 理论上要输出每个 token 位置对应值，但实际上是对整个 seq 打分。在 deepspeed-chat 里设计为，把 reward model 输出的 reward 加到最后一个 token，而其他位置的 token reward 为 0。

注：因此，γ在 RLHF 中通常设置为 1，之前有个问题是，设置为 1 不就等于没折扣了吗？其实是和 reward 设计一致，因为只有最后一步加上 reward model 输出的 reward。若引入时间折扣，更靠近结尾的 token 对 return 影响更大，可能偏向更短的回答，而 RLHF 通常希望只根据最终质量给分，不强调“越快越好”。

这里有一个工程细节，写进优化目标的 KL 散度限制，会作为一种 reward 信号，给每个 token 打分。

来看看具体 reward 值计算

DeepSpeedExamples/applications/DeepSpeed-Chat/dschat/rlhf/ppo_trainer.py 第 181 行起

```
def compute_rewards(self, prompts, log_probs, ref_log_probs, reward_score,
                    action_mask):

        kl_divergence_estimate = -self.kl_ctl * (log_probs - ref_log_probs)
        rewards = kl_divergence_estimate
        start = prompts.shape[1] - 1
        ends = start + action_mask[:, start:].sum(1) + 1
        reward_clip = torch.clamp(reward_score, -self.clip_reward_value,
                                  self.clip_reward_value)
        batch_size = log_probs.shape[0]
        for j in range(batch_size):
```

```
            rewards[j, start:ends[j]][-1] += reward_clip[j]

        return rewards
```

- $r_t(\theta)$是什么？

直接看公式

$$r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{old}}(a_t|s_t)$$

这里涉及到重要性采样, 理想情况下生产一条数据更新一遍模型, 完全 on policy。但通常生产一批数据模型会更新多次，我们要利用旧策略$\pi_{\theta_{old}}$收集的数据来更新新策略。 $r(\theta)$像是"修正因子"，把 $\pi_{\theta_{old}}$的采样结果校准到 π_{θ}的分布上。

clip 是控制更新幅度。

以上可得到最终 Loss 函数。

DeepSpeedExamples/applications/DeepSpeed-Chat/dschat/rlhf/ppo_trainer.py 第 282 行起

```
    def actor_loss_fn(self, logprobs, old_logprobs, advantages, mask):
        ## policy gradient loss
        log_ratio = (logprobs - old_logprobs) * mask
        ratio = torch.exp(log_ratio)
        pg_loss1 = -advantages * ratio
        pg_loss2 = -advantages * torch.clamp(ratio, 1.0 - self.cliprange,
                                             1.0 + self.cliprange)
        pg_loss = torch.sum(torch.max(pg_loss1, pg_loss2) * mask) / mask.sum()
        return pg_loss
```

参考

<https://github.com/deepspeedai/DeepSpeedExamples>

<https://zhuanlan.zhihu.com/p/624797778>

<https://lishizheng.blog.csdn.net/article/details/145826166>

《强化学习的数学原理（英文版）》西湖大学赵世钰【推一下赵老师，虽然因个人懒惰和方向问题学的较少，

但犹记得一年多前看这本书和听老师讲课时不断在感慨这本书这门课的精妙】